```
# 02_eda_and_cache.py
# Exploratory analysis, band-order check, NDVI engineering, train/val/test split, and a
# compact cache (128 x 128, channels R, G, B, NIR, NDVI) that the training script reads.
# Run each "# %%" block as its own Colab cell. No GPU needed.

# %% [1] Imports and settings
import os, json
import numpy as np
import pandas as pd
import cv2
import tifffile
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from concurrent.futures import ThreadPoolExecutor
from sklearn.model_selection import train_test_split

ROOT = os.environ.get("AMAZON_ROOT", "/content/drive/MyDrive/amazon_project")
try:
    from google.colab import drive
    drive.mount("/content/drive")
except ImportError:
    pass

SIZE = 128        # tiles are resized from 256 to 128 to make training ~4x faster
SUBSET = None     # e.g. 15000 for a quick run (keeps ALL rare-label tiles); None = all
SEED = 42
FIG, CACHE = f"{ROOT}/figs", f"{ROOT}/cache"
os.makedirs(FIG, exist_ok=True)
os.makedirs(CACHE, exist_ok=True)
paths = json.load(open(f"{ROOT}/paths.json"))

# %% [2] Load labels and build the 17-column label matrix
df = pd.read_csv(paths["csv"])
tag_lists = df["tags"].str.split()
LABELS = sorted({t for tl in tag_lists for t in tl})
assert len(LABELS) == 17, LABELS
Y = np.zeros((len(df), 17), dtype=np.uint8)
for i, tl in enumerate(tag_lists):
    for t in tl:
        Y[i, LABELS.index(t)] = 1
names = df["image_name"].tolist()
print("Tiles:", len(df), "| labels:", LABELS)

# %% [3] Label frequency (this is your class-imbalance evidence for the slides)
counts = pd.Series(Y.sum(0), index=LABELS).sort_values(ascending=False)
table = pd.DataFrame({"tiles": counts, "percent": (100 * counts / len(df)).round(2)})
table.to_csv(f"{FIG}/label_counts.csv")
print(table)

fig, ax = plt.subplots(figsize=(10, 5))
cols = ["#FF8A3D" if p < 1 else "#3FB6A8" for p in table["percent"]]
ax.bar(table.index, table["tiles"], color=cols)
ax.set_yscale("log")
ax.set_ylabel("Number of tiles (log scale)")
ax.set_title("Label frequency (orange = under 1% of tiles)")
plt.xticks(rotation=60, ha="right")
plt.tight_layout()
plt.savefig(f"{FIG}/fig_label_counts.png", dpi=200)
plt.close()

co = Y.T.astype(float) @ Y.astype(float)
co = co / np.diag(co)[:, None]  # P(column label | row label)
fig, ax = plt.subplots(figsize=(8, 7))
im = ax.imshow(co, cmap="Greens", vmin=0, vmax=1)
ax.set_xticks(range(17))
```

```
ax.set_xticklabels(LABELS, rotation=70, ha="right")
ax.set_yticks(range(17))
ax.set_yticklabels(LABELS)
ax.set_title("Label co-occurrence: P(column | row)")
plt.colorbar(im)
plt.tight_layout()
plt.savefig(f"{FIG}/fig_label_cooccurrence.png", dpi=200)
plt.close()
```

```
# %% [4] Optional subset (keeps every tile that has a rare label)
rng = np.random.RandomState(SEED)
keep = np.arange(len(df))
if SUBSET and SUBSET < len(df):
    rare_idx = [LABELS.index(l) for l in LABELS if counts[l] < 1000]
    is_rare = Y[:, rare_idx].any(1)
    others = np.where(~is_rare)[0]
    need = max(SUBSET - int(is_rare.sum()), 0)
    keep = np.sort(np.concatenate([np.where(is_rare)[0], rng.choice(others, need, False)]))
Y = Y[keep]
names = [names[i] for i in keep]
N = len(names)
print("Tiles used:", N)


def tif_path(n):
    return os.path.join(paths["tif_dir"], n + ".tif")


def read_tif(n):
    a = tifffile.imread(tif_path(n)).astype(np.float32)
    if a.ndim == 3 and a.shape[0] == 4 and a.shape[-1] != 4:
        a = np.transpose(a, (1, 2, 0))
    return a  # (256, 256, 4)
```

```
# %% [5] Check the band order (the TIF band order is easy to get wrong)
# The Planet TIFs are normally stored as B, G, R, NIR. We verify it with physics: moving from
# forest to cleared land, the RED band brightens much more than the BLUE band, and NDVI must
# be highest for forest and lowest for water. If the data disagrees, we switch to R, G, B, NIR.
samp = rng.choice(N, min(600, N), replace=False)
raw = np.stack([read_tif(names[i]) for i in samp])           # (S, 256, 256, 4)
tile_mean = raw.reshape(len(samp), -1, 4).mean(1)             # (S, 4)
ys = Y[samp]
prim = ys[:, LABELS.index("primary")] == 1
open_ = (ys[:, LABELS.index("agriculture")] == 1) | (ys[:, LABELS.index("bare_ground")] == 1)
prim, open_ = prim & ~open_, open_ & ~prim
ratio = tile_mean[open_].mean(0) / tile_mean[prim].mean(0)    # brightening per band
print("Cleared/forest brightness ratio per band index 0..3:", ratio.round(2))
orders = {"BGRN": dict(b=0, g=1, r=2, nir=3), "RGBN": dict(b=2, g=1, r=0, nir=3)}
ORDER = "BGRN"
if ratio[0] > 1.15 * ratio[2]:       # band 0 brightens much more than band 2 -> band 0 is red
    ORDER = "RGBN"
O = orders[ORDER]
ndvi_t = (tile_mean[:, O["nir"]] - tile_mean[:, O["r"]]) / (
    tile_mean[:, O["nir"]] + tile_mean[:, O["r"]] + 1e-6)
for l in ["primary", "agriculture", "water"]:
    m = ys[:, LABELS.index(l)] == 1
    if m.sum():
        print(f"mean NDVI for {l}: {ndvi_t[m].mean():.3f}")
print("Chosen band order:", ORDER, "-> forest NDVI should be highest, water lowest")
json.dump({"order": ORDER, **O}, open(f"{CACHE}/order.json", "w"))

# Percentile stretch limits per band, from the sample
lo = np.percentile(raw.reshape(-1, 4), 1, axis=0)
hi = np.percentile(raw.reshape(-1, 4), 99, axis=0)
json.dump({"lo": lo.tolist(), "hi": hi.tolist()}, open(f"{CACHE}/band_stats.json", "w"))
```

```
# Visual check: forest should look green, cleared land tan/brown. If colours look wrong
# (e.g. blue forest), the band order is wrong: set ORDER manually above.
fig, axes = plt.subplots(2, 4, figsize=(10, 5))
for r, grp in enumerate([np.where(prim)[0][:4], np.where(open_)[0][:4]]):
    for c, k in enumerate(grp):
        img = np.stack([np.clip((raw[k][..., O[x]] - lo[O[x]]) / (hi[O[x]] - lo[O[x]] + 1e-6),
                                0, 1) for x in ("r", "g", "b")], -1) ** 0.8
        axes[r][c].imshow(img)
        axes[r][c].axis("off")
        axes[r][c].set_title(["forest", "cleared"][r], fontsize=9)
plt.tight_layout()
plt.savefig(f"{FIG}/fig_band_check.png", dpi=150)
plt.close()
del raw
```

 # %% [6] Build the cache: channels = [R, G, B, NIR, NDVI], float16, shape (N, 5, S, S)

```
Xmm = np.lib.format.open_memmap(f"{CACHE}/X.npy", mode="w+", dtype=np.float16,
                                shape=(N, 5, SIZE, SIZE))


def process(i):
    a = read_tif(names[i])
    red, nir = a[..., O["r"]], a[..., O["nir"]]
    ndvi = (nir - red) / (nir + red + 1e-6)                    # in [-1, 1]
    chans = []
    for src in (O["r"], O["g"], O["b"], O["nir"]):
        chans.append(np.clip((a[..., src] - lo[src]) / (hi[src] - lo[src] + 1e-6), 0, 1))
    chans.append((np.clip(ndvi, -1, 1) + 1) / 2)               # NDVI mapped to [0, 1]
    arr = np.stack(chans, -1).astype(np.float32)
    arr = cv2.resize(arr, (SIZE, SIZE), interpolation=cv2.INTER_AREA)
    Xmm[i] = np.transpose(arr, (2, 0, 1)).astype(np.float16)


with ThreadPoolExecutor(8) as ex:
    for k, _ in enumerate(ex.map(process, range(N))):
        if k % 2000 == 0:
            print(f"cached {k}/{N}")
Xmm.flush()
np.save(f"{CACHE}/Y.npy", Y)
json.dump(LABELS, open(f"{CACHE}/labels.json", "w"))
json.dump(names, open(f"{CACHE}/names.json", "w"))
```

 # %% [7] Train / validation / test split (70 / 15 / 15)

```
# Stratify on each tile's rarest label so every split contains the rare classes.
freq = Y.sum(0)
strata = np.array([min(np.where(y == 1)[0], key=lambda j: freq[j]) for y in Y])
idx = np.arange(N)
tr, tmp = train_test_split(idx, test_size=0.30, stratify=strata, random_state=SEED)
va, te = train_test_split(tmp, test_size=0.50, stratify=strata[tmp], random_state=SEED)
np.savez(f"{CACHE}/splits.npz", train=np.sort(tr), val=np.sort(va), test=np.sort(te))
print("train/val/test:", len(tr), len(va), len(te))
print("rare-label counts in test:",
      {l: int(Y[te][:, LABELS.index(l)].sum()) for l in LABELS if freq[LABELS.index(l)] < 400})
```

 # %% [8] Per-channel mean/std from the training split (used to normalise during training)

```
Xr = np.load(f"{CACHE}/X.npy", mmap_mode="r")
pick = np.sort(rng.choice(tr, min(3000, len(tr)), replace=False))
sub = np.asarray(Xr[pick]).astype(np.float32)
mean, std = sub.mean((0, 2, 3)), sub.std((0, 2, 3)) + 1e-6
np.savez(f"{CACHE}/stats.npz", mean=mean, std=std)
print("channel means:", mean.round(3), "stds:", std.round(3))
print("Done. Next: 03_real_figures.py (optional but recommended) and 04_train_models.py")
```